

Study Notes: Formal Languages and Automata Theory

Topics 2.9 to 4.2

Ivan Raphael Jaison

Disclaimer: This entire document is AI-generated and intended for study purposes. Please verify all theories, concepts, and proofs independently.

Contents

1	Module 2: Properties of Regular Languages	2
1.1	2.9 The Pumping Lemma for Regular Languages	2
1.2	2.10 Pumping Lemma as a Tool to Prove Non-Regularity	3
2	Module 3: Context-Free Languages and Grammars	5
2.1	3.1 & 3.2 Formal Definition and Design of CFG	5
2.2	3.3 & 3.4 Parse Trees, Ambiguity, and Programming Languages	6
2.3	3.5 & 3.6 Simplification of Context-Free Languages	8
2.4	3.7 & 3.8 Normal Forms (Chomsky and Greibach)	9
2.5	3.9 & 3.10 Pumping Lemma for CFLs and Closure Properties	11
3	Module 4: Pushdown Automata and Context Sensitive Languages	13
3.1	4.1 Pushdown Automata (Formal Definition)	13
3.2	4.2 DPDA and NPDA / Equivalence Modalities	15
	3.2.1 Non-Equivalence of DPDA and NPDA	15
	3.2.2 Equivalence of Final State and Empty Stack Acceptance	15

1 Module 2: Properties of Regular Languages

1.1 2.9 The Pumping Lemma for Regular Languages

The Pumping Lemma is a powerful mathematical tool used to prove that a specific language is **not** regular. It states that all regular languages have a particular property: any sufficiently long string in the language can be broken into three parts, such that the middle part can be repeated ("pumped") any number of times, and the resulting string will still belong to the language.

Theorem 1.1 (The Pumping Lemma for Regular Languages). *Let L be a regular language. Then there exists a constant $p \geq 1$ (called the pumping length) such that any string $s \in L$ with $|s| \geq p$ can be divided into three pieces, $s = xyz$, satisfying the following three conditions:*

1. $xy^iz \in L$ for every $i \geq 0$
2. $|y| > 0$ (i.e., $y \neq \epsilon$)
3. $|xy| \leq p$

Formal Proof. Since L is a regular language, there exists a Deterministic Finite Automaton (DFA) $M = (Q, \Sigma, \delta, q_0, F)$ that accepts L . Let p be the number of states in Q (i.e., $p = |Q|$).

Consider any string $s = a_1a_2 \dots a_n$ in L such that $n \geq p$. Let $q_0, q_1, \dots, q_n$ be the sequence of states that M enters while processing s , where q_0 is the start state and $q_i = \delta(q_{i-1}, a_i)$. Since $s \in L$, q_n must be an accepting state ($q_n \in F$).

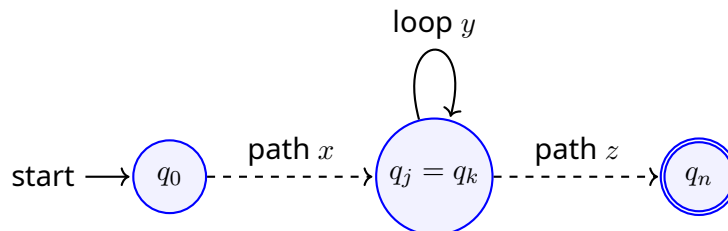
The sequence of states $q_0, q_1, \dots, q_p$ consists of exactly $p + 1$ states. Since there are only p unique states in M , by the **Pigeonhole Principle**, at least one state must repeat in this sequence. Let this repeated state occur at indices j and k , where $0 \leq j < k \leq p$. Thus, $q_j = q_k$.

We can now divide the string s into three parts:

- $x = a_1 \dots a_j$ (the path from q_0 to q_j)
- $y = a_{j+1} \dots a_k$ (the path from q_j back to $q_k = q_j$, forming a loop)
- $z = a_{k+1} \dots a_n$ (the path from q_k to the final state q_n)

Since $q_j = q_k$, the automaton can loop around the path y zero times ($i = 0$), once ($i = 1$), or multiple times ($i > 1$), and it will still eventually read z and end up in the accepting state q_n . Therefore, $xy^iz \in L$ for all $i \geq 0$.

Furthermore, since $j < k$, the length of y is strictly greater than 0 ($|y| > 0$). Lastly, since the repetition occurs within the first $p + 1$ states, the combined length of x and y is at most p ($|xy| \leq p$). $\square$



1.2 2.10 Pumping Lemma as a Tool to Prove Non-Regularity

To prove a language L is not regular using the Pumping Lemma, we use a proof by contradiction (often modeled as an adversarial game): 1. Assume L is regular. 2. The Pumping Lemma states there must be a pumping length p . 3. We **choose** a specific string $s \in L$ where $|s| \geq p$. 4. The adversary divides $s = xyz$ subject to $|xy| \leq p$ and $|y| > 0$. 5. We **choose** an $i \geq 0$ such that $xy^iz \notin L$. 6. This contradicts the lemma, proving our initial assumption wrong. Hence, L is not regular.

Practice Problems for 2.9 & 2.10

Practice Problem 1.1. Prove that the language $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

Solution:

Assume for the sake of contradiction that L is regular. Let p be the pumping length given by the Pumping Lemma.

Choose the string $s = a^p b^p$. Clearly, $s \in L$ and $|s| = 2p \geq p$.

By the Pumping Lemma, s can be written as $s = xyz$ such that $|y| > 0$, $|xy| \leq p$, and $xy^iz \in L$ for all $i \geq 0$.

Since $|xy| \leq p$, the substring xy must consist entirely of a 's. Therefore, $y = a^k$ for some $k > 0$.

Now, pump y by choosing $i = 2$ (or $i = 0$). The pumped string is $xy^2z = a^{p+k}b^p$.

Since $k > 0$, the number of a 's ($p + k$) is not equal to the number of b 's (p). Thus, $xy^2z \notin L$.

This contradicts the Pumping Lemma. Therefore, L is not regular.

Practice Problem 1.2. Prove that the language $L = \{w \in \{0, 1\}^* \mid \text{number of 0s equals number of 1s}\}$ is not regular.

Solution:

Assume L is regular. Let p be the pumping length.

Choose $s = 0^p 1^p$. Since s has equal numbers of 0s and 1s, $s \in L$, and $|s| = 2p \geq p$.

According to the Pumping Lemma, $s = xyz$ with $|xy| \leq p$ and $|y| > 0$.

Because xy is confined to the first p characters, y consists only of 0s ($y = 0^k, k > 0$).

Pump with $i = 0$: $xy^0z = xz$. This string has $p - k$ zeros and p ones.

Since $k > 0$, the number of 0s does not equal the number of 1s, so $xz \notin L$.

This is a contradiction. Hence, L is not regular.

Practice Problem 1.3. Prove that $L = \{a^m \mid m \text{ is a prime number}\}$ is not regular.

Solution:

Assume L is regular with pumping length p .

Choose a prime $q \geq p$. Let $s = a^q$. Since $q \geq p$, we can split $s = xyz$ with $|xy| \leq p$, $|y| > 0$, and $xy^iz \in L$ for all $i \geq 0$.

Let $|y| = k$ (where $k > 0$) and $|xz| = q - k$.

By the lemma, xy^iz has length $|xz| + i|y| = (q - k) + ik$.

We need to find an i such that $(q - k) + ik$ is composite.

Let $i = q + 1$. The length becomes:

$(q - k) + (q + 1)k = q - k + qk + k = q + qk = q(1 + k)$.

Since q is a prime ($q \geq 2$) and $k \geq 1$ (so $1 + k \geq 2$), $q(1 + k)$ is a product of two integers greater than 1, meaning it is a composite number.

Therefore, the string $xy^{q+1}z$ has a composite length, so it does not belong to L . This contradicts the Pumping Lemma. L is not regular.

2 Module 3: Context-Free Languages and Grammars

2.1 3.1 & 3.2 Formal Definition and Design of CFG

A **Context-Free Grammar (CFG)** is a mathematical system for generating strings in a formal language. It is defined as a 4-tuple $G = (V, T, P, S)$, where:

- V : A finite set of variables (or non-terminals).
- T : A finite set of terminals (alphabet of the language).
- P : A finite set of production rules of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.
- S : The start variable ($S \in V$).

Derivations: A sequence of applications of production rules that generates a string.

- **Leftmost Derivation (LMD):** At each step, the leftmost variable in the string is replaced.
 - **Rightmost Derivation (RMD):** At each step, the rightmost variable in the string is replaced.
-

Practice Problems for 3.1 & 3.2

Practice Problem 2.1. Design a CFG for the language of palindromes over the alphabet $\{0, 1\}$. Generate the string 01010 using a Leftmost Derivation.

Solution:

A palindrome reads the same forwards and backwards. The grammar G has rules: $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \epsilon$

Leftmost derivation for 01010: $S \Rightarrow 0S0 \Rightarrow 01S10 \Rightarrow 01010$

Practice Problem 2.2. Design a CFG for the language $L = \{a^n b^{2n} \mid n \geq 1\}$.

Solution:

For every single 'a' added to the left, we must add two 'b's to the right. The base case for $n = 1$ is abb . The grammar is: $S \rightarrow aSbb \mid abb$

Practice Problem 2.3. Design a CFG for the language $L = \{w \in \{a, b\}^* \mid n_a(w) = n_b(w)\}$, where $n_x(w)$ is the number of x 's in w .

Solution:

The grammar must ensure that every 'a' is balanced by a 'b'. The rules are: $S \rightarrow SS \mid aSb \mid bSa \mid \epsilon$

2.2 3.3 & 3.4 Parse Trees, Ambiguity, and Programming Languages

A **Parse Tree** is a graphical representation of a derivation. The root is the start symbol S , interior nodes are variables, and leaves are terminals or ϵ .

Ambiguity: A CFG G is ambiguous if there exists at least one string in $L(G)$ that has **more than one valid parse tree** (or equivalently, more than one Leftmost Derivation).

In programming languages, CFGs define syntax. Ambiguity is problematic because it implies a program could have multiple meanings (e.g., $3 + 4 * 5$ evaluating to 35 or 23). We resolve ambiguity by enforcing precedence and associativity through grammatical structuring.

Practice Problems for 3.3 & 3.4

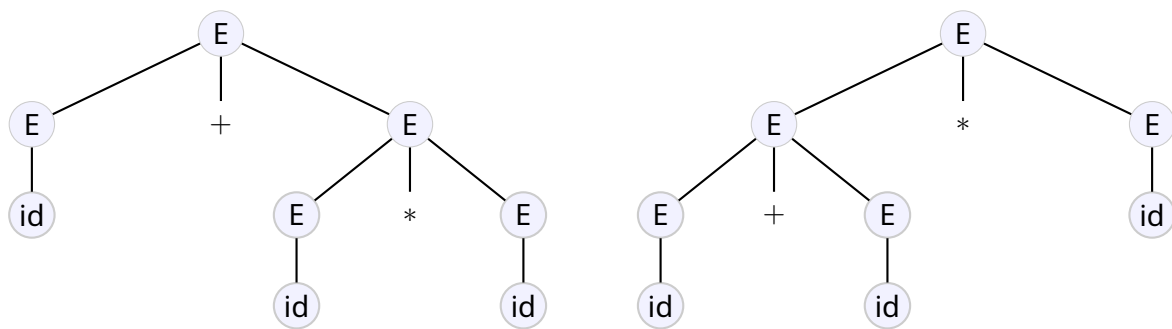
Practice Problem 2.4. Consider the grammar $E \rightarrow E + E \mid E * E \mid id$. Show that the grammar is ambiguous for the string $id + id * id$.

Solution:

We can show two distinct Leftmost Derivations for the same string:

LMD 1 (giving precedence to +): $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$

LMD 2 (giving precedence to *): $E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$



Since there are two distinct LMDs for the same string, the grammar is ambiguous.

Practice Problem 2.5. Rewrite the grammar from the previous problem to remove the ambiguity, giving $*$ higher precedence than $+$, and making both left-associative.

Solution:

To establish precedence, we introduce new variables (Term T for multiplication, Factor F for the base elements):

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow id$$

Practice Problem 2.6. Show that the "dangling-else" grammar $S \rightarrow \text{if } b \text{ then } S \mid \text{if } b \text{ then } S \text{ else } S \mid a$ is ambiguous.

Solution:

Consider the string: `if b then if b then a else a`

It has two parse trees depending on which "if" the "else" binds to:

Derivation 1 (binds to outer if): $S \Rightarrow \text{if } b \text{ then } S \text{ else } S \Rightarrow \text{if } b \text{ then } (\text{if } b \text{ then } a) \text{ else } a$

Derivation 2 (binds to inner if): $S \Rightarrow \text{if } b \text{ then } S \Rightarrow \text{if } b \text{ then } (\text{if } b \text{ then } a \text{ else } a)$

Because multiple parse trees exist, the grammar is ambiguous.

2.3 3.5 & 3.6 Simplification of Context-Free Languages

Simplification of a CFG involves three main steps applied in a specific order:

1. **Eliminate ϵ -productions** ($A \rightarrow \epsilon$).
 2. **Eliminate Unit productions** ($A \rightarrow B$).
 3. **Eliminate Useless symbols** (symbols that do not generate terminals, or are unreachable from S).
-

Practice Problems for 3.5 & 3.6

Practice Problem 2.7. *Eliminate useless symbols from the grammar: $S \rightarrow AB \mid a, A \rightarrow b, B \rightarrow bB, C \rightarrow c$*

Solution:

Step 1: Identify generating symbols (variables that can eventually derive a terminal string).

$A \rightarrow b$ (A is generating).

$S \rightarrow a$ (S is generating).

$B \rightarrow bB$ (B never terminates, so B is non-generating).

$C \rightarrow c$ (C is generating).

Remove non-generating B and all productions involving it.

Grammar becomes: $S \rightarrow a, A \rightarrow b, C \rightarrow c$.

Step 2: Identify reachable symbols from S .

Only S is reachable. A and C are unreachable.

Final simplified grammar:

$S \rightarrow a$

Practice Problem 2.8. *Eliminate ϵ -productions from: $S \rightarrow ABaC, A \rightarrow BC, B \rightarrow b \mid \epsilon, C \rightarrow D \mid \epsilon, D \rightarrow d$*

Solution:

First, find all nullable variables.

$B \rightarrow \epsilon, C \rightarrow \epsilon$.

Since $A \rightarrow BC$, and both B and C are nullable, A is also nullable.

Nullable variables: $\{A, B, C\}$.

Now, add productions by dropping combinations of nullable variables in the right-hand sides, and remove explicit ϵ -productions:

$S \rightarrow ABaC \mid BaC \mid AaC \mid ABa \mid aC \mid Ba \mid Aa \mid a$

$A \rightarrow BC \mid B \mid C$

$B \rightarrow b$

$C \rightarrow D$

$D \rightarrow d$

Practice Problem 2.9. *Eliminate unit productions from: $S \rightarrow A \mid bb, A \rightarrow B \mid b, B \rightarrow S \mid a$*

Solution:

Find unit pairs:

(S, A) (since $S \rightarrow A$)

(S, B) (since $S \rightarrow A \rightarrow B$)
 (S, S) (since $S \rightarrow A \rightarrow B \rightarrow S$)
 (A, B) (since $A \rightarrow B$)
 (A, S) (since $A \rightarrow B \rightarrow S$)
 (B, S) (since $B \rightarrow S$)
 (B, A) (since $B \rightarrow S \rightarrow A$)

For each pair (X, Y) , add rules $X \rightarrow \alpha$ where $Y \rightarrow \alpha$ is a non-unit production.

Non-unit productions are: $S \rightarrow bb$, $A \rightarrow b$, $B \rightarrow a$.

New grammar:

$S \rightarrow bb \mid b \mid a$

$A \rightarrow b \mid a \mid bb$

$B \rightarrow a \mid bb \mid b$

2.4 3.7 & 3.8 Normal Forms (Chomsky and Greibach)

Chomsky Normal Form (CNF): A CFG is in CNF if all productions are of the form $A \rightarrow BC$ or $A \rightarrow a$, where A, B, C are variables and a is a terminal.

Greibach Normal Form (GNF): A CFG is in GNF if all productions are of the form $A \rightarrow a\alpha$, where a is a terminal and $\alpha \in V^*$ (a string of zero or more variables).

Practice Problems for 3.7 & 3.8

Practice Problem 2.10. Convert the following grammar to Chomsky Normal Form (CNF): $S \rightarrow aSb \mid ab$

Solution:

First, create variables for terminals: Let $X_a \rightarrow a$ and $X_b \rightarrow b$.

Rewrite productions:

$S \rightarrow X_a S X_b \mid X_a X_b$

Now, restrict the RHS to two variables. For $S \rightarrow X_a S X_b$, introduce $Y_1 \rightarrow S X_b$.

Final CNF grammar:

$S \rightarrow X_a Y_1 \mid X_a X_b$

$Y_1 \rightarrow S X_b$

$X_a \rightarrow a$

$X_b \rightarrow b$

Practice Problem 2.11. Convert to CNF: $S \rightarrow AB$, $A \rightarrow aA \mid a$, $B \rightarrow bB \mid b$.

Solution:

$S \rightarrow AB$ is already valid.

For $A \rightarrow aA$, create $X_a \rightarrow a$, making it $A \rightarrow X_a A$.

$A \rightarrow a$ is valid.

For $B \rightarrow bB$, create $X_b \rightarrow b$, making it $B \rightarrow X_b B$.

$B \rightarrow b$ is valid.

Final CNF:

$S \rightarrow AB$, $A \rightarrow X_a A \mid a$, $B \rightarrow X_b B \mid b$, $X_a \rightarrow a$, $X_b \rightarrow b$.

Practice Problem 2.12. Convert the following grammar to Greibach Normal Form (GNF): $S \rightarrow AB, A \rightarrow a, B \rightarrow b$

Solution:

GNF requires all rules to start with exactly one terminal.

$A \rightarrow a$ and $B \rightarrow b$ are already in GNF.

In $S \rightarrow AB$, the RHS starts with a variable A . We substitute the rules of A into S .

Since $A \rightarrow a$, we replace A in S to get: $S \rightarrow aB$.

Final GNF:

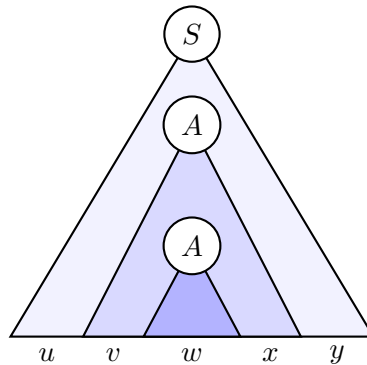
$S \rightarrow aB, A \rightarrow a, B \rightarrow b$.

2.5 3.9 & 3.10 Pumping Lemma for CFLs and Closure Properties

Theorem 2.1 (Pumping Lemma for CFLs). *For every Context-Free Language L , there exists a constant p such that every string $s \in L$ where $|s| \geq p$ can be written as $s = uvwxy$, satisfying:*

1. $uv^iwx^iy \in L$ for all $i \geq 0$
2. $|vx| > 0$
3. $|vwx| \leq p$

Proof Sketch: In a grammar in CNF, a parse tree for a long string will be very deep. By the Pigeonhole Principle, some variable A must repeat on a path from the root to a leaf. The upper A derives vAy , and the lower A derives w . Pumping occurs by repeating the derivation $A \Rightarrow^* vAy$.



Closure Properties of CFLs:

- **Closed under:** Union, Concatenation, Kleene Star.
- **NOT closed under:** Intersection, Complement.

Practice Problems for 3.9 & 3.10

Practice Problem 2.13. *Prove that $L = \{a^n b^n c^n \mid n \geq 0\}$ is not a Context-Free Language.*

Solution:

Assume L is a CFL. Let p be the pumping length. Choose $s = a^p b^p c^p \in L$. $|s| = 3p \geq p$.

By the lemma, $s = uvwxy$ where $|vwx| \leq p$ and $|vx| > 0$.

Because $|vwx| \leq p$, the substring vwx can span at most two different types of symbols (either only a 's, only b 's, only c 's, a 's and b 's, or b 's and c 's). It cannot contain a 's, b 's, and c 's simultaneously.

Thus, v and x contain at most two types of symbols.

If we pump $i = 2$ (uv^2wx^2y), we increase the count of at most two types of symbols, leaving the third un-incremented. This breaks the equality of the number of a 's, b 's, and c 's.

Therefore, $uv^2wx^2y \notin L$. Contradiction. L is not a CFL.

Practice Problem 2.14. *Prove that $L = \{ww \mid w \in \{a, b\}^*\}$ is not context-free.*

Solution:

Assume L is a CFL. Let p be the pumping length. Choose $s = a^p b^p a^p b^p$.

By the Pumping lemma, $s = uvwxy$ with $|vwx| \leq p$ and $|vx| > 0$.

Because $|vwx| \leq p$, vwx can lie entirely within the first half, entirely within the second half, or straddle the middle.

In any case, pumping $i = 0$ (removing v and x) removes characters from at most a length p segment. This will alter one of the a^p or b^p blocks without equivalently altering its corresponding pair block in the other half of the string.

Thus uv^0wx^0y cannot be written in the form ww . Contradiction. L is not a CFL.

Practice Problem 2.15. *Using closure properties, show why Context-Free Languages are not closed under intersection.*

Solution:

Consider two languages: $L_1 = \{a^n b^n c^m \mid n, m \geq 0\}$ (CFL, grammar matches a and b , leaves c arbitrary) $L_2 = \{a^m b^n c^n \mid m, n \geq 0\}$ (CFL, grammar matches b and c , leaves a arbitrary) Both L_1 and L_2 are context-free. Their intersection is $L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$. As proved in Problem 3.9.1, $\{a^n b^n c^n \mid n \geq 0\}$ is NOT a context-free language. Since the intersection of two CFLs yielded a non-CFL, the class of Context-Free Languages is not closed under intersection.

3 Module 4: Pushdown Automata and Context Sensitive Languages

3.1 4.1 Pushdown Automata (Formal Definition)

A **Pushdown Automaton (PDA)** is a finite automaton equipped with a stack, giving it the memory capability to recognize context-free languages. Formally, a PDA is a 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$:

- Q : Finite set of states.
- Σ : Input alphabet.
- Γ : Stack alphabet.
- δ : Transition function $Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$.
- $q_0 \in Q$: Start state.
- $Z_0 \in \Gamma$: Initial stack symbol.
- $F \subseteq Q$: Set of accepting/final states.

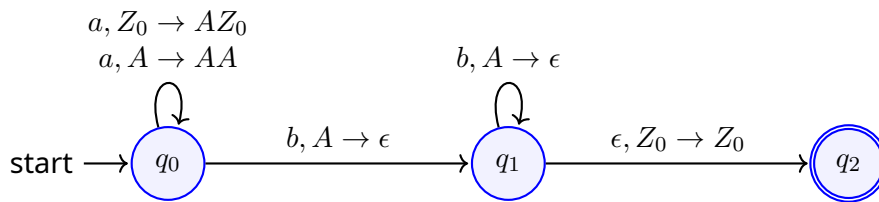
Practice Problems for 4.1

Practice Problem 3.1. Design a PDA for $L = \{a^n b^n \mid n \geq 1\}$ by final state.

Solution:

Strategy: Push 'A' onto the stack for every 'a' read. For every 'b' read, pop an 'A'. Let $M = (\{q_0, q_1, q_2\}, \{a, b\}, \{Z_0, A\}, \delta, q_0, Z_0, \{q_2\})$. Transitions:

1. $\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$ (Push first a)
2. $\delta(q_0, a, A) = \{(q_0, AA)\}$ (Push subsequent a's)
3. $\delta(q_0, b, A) = \{(q_1, \epsilon)\}$ (Read first b, pop A, transition to q_1)
4. $\delta(q_1, b, A) = \{(q_1, \epsilon)\}$ (Read subsequent b's, pop A)
5. $\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$ (If input empty and stack only has Z_0 , go to final state q_2)



Practice Problem 3.2. Design a PDA for $L = \{wcw^R \mid w \in \{a, b\}^*\}$.

Solution:

Strategy: Push symbols until 'c' is reached. After 'c', pop and match symbols. Transitions:

1. $\delta(q_0, a, X) = \{(q_0, aX)\}$ for all $X \in \{Z_0, a, b\}$ (Push a)
2. $\delta(q_0, b, X) = \{(q_0, bX)\}$ for all $X \in \{Z_0, a, b\}$ (Push b)

3. $\delta(q_0, c, X) = \{(q_1, X)\}$ for all $X \in \{Z_0, a, b\}$ (Switch states on c)
4. $\delta(q_1, a, a) = \{(q_1, \epsilon)\}$ (Match a)
5. $\delta(q_1, b, b) = \{(q_1, \epsilon)\}$ (Match b)
6. $\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$ (Accept by final state q_2)

Practice Problem 3.3. Design a PDA for balanced parentheses $L = \{w \in \{(,)\}^* \mid w \text{ is balanced}\}$.

Solution:

Strategy: Push 'X' for '('. Pop 'X' for ')'. Transitions:

1. $\delta(q_0, (, Z_0) = \{(q_0, XZ_0)\}$
2. $\delta(q_0, (, X) = \{(q_0, XX)\}$
3. $\delta(q_0,), X) = \{(q_0, \epsilon)\}$
4. $\delta(q_0, \epsilon, Z_0) = \{(q_f, Z_0)\}$ (Accept on empty input and initial stack)

3.2 4.2 DPDA and NPDA / Equivalence Modalities

Important Note regarding the Syllabus: The syllabus mentions "Equivalence DPDA and NPDA (proof required), conversions in both Directions". In automata theory, DPDA (Deterministic PDA) and NPDA (Nondeterministic PDA) are **strictly NOT equivalent**. NPDAs accept all Context-Free Languages, while DPDAs only accept a subset known as Deterministic Context-Free Languages (DCFLs). It is impossible to convert every NPDA to a DPDA. However, there is an equivalence regarding **Acceptance by Final State vs. Acceptance by Empty Stack** for PDAs, which involves conversions in both directions. We address both the non-equivalence of DPDA/NPDA and the structural equivalences of PDAs below.

3.2.1 Non-Equivalence of DPDA and NPDA

Theorem 3.1. *NPDAs are strictly more powerful than DPDAs.*

Proof. To prove they are not equivalent, we must show a language accepted by an NPDA but not by any DPDA. Consider the language of even-length palindromes $L = \{ww^R \mid w \in \{a, b\}^*\}$.

An NPDA easily accepts this by nondeterministically "guessing" the middle of the string and transitioning from pushing to popping.

However, a DPDA processes input sequentially and deterministically. It has no way of knowing when the midpoint of an arbitrary string occurs without a marker (like the 'c' in wcw^R). By the time it reads the second half, if it hasn't switched states to pop, it cannot backtrack. Thus, no DPDA can accept L . Because L is a CFL (accepted by NPDA) but not a DCFL (not accepted by DPDA), DPDA and NPDA are not equivalent. $\square$

3.2.2 Equivalence of Final State and Empty Stack Acceptance

Any language accepted by a PDA by Final State ($L(M_f)$) can be accepted by an equivalent PDA by Empty Stack ($N(M_e)$), and vice-versa.

- **Final State to Empty Stack:** Add a new start state p_0 and new stack bottom X_0 . Add ϵ -transitions from all final states in M_f to a new state p_{erase} , where all stack symbols are popped until the stack is empty.
- **Empty Stack to Final State:** Add a new start state p_0 , new stack bottom X_0 , and a new final state p_f . Whenever X_0 is exposed at the top of the stack (meaning the original PDA emptied its stack), ϵ -transition to p_f to accept.

Practice Problems for 4.2

Practice Problem 3.4. *Explain why $L = \{a^n b^n \mid n \geq 1\} \cup \{a^n b^{2n} \mid n \geq 1\}$ is accepted by an NPDA but not a DPDA.*

Solution:

An NPDA can use an ϵ -transition at the start state to "guess" whether it should check for b^n (pop one A for each b) or b^{2n} (pop one A for every two b 's).

A DPDA cannot make this guess. If it sees a^n , it pushes n symbols. Once it starts reading b 's, it must commit to a matching rule without knowing if the string will end after n or $2n$ copies of b . This ambiguity cannot be resolved deterministically without reading ahead, thus no DPDA exists for L .

Practice Problem 3.5. Let M be a PDA accepting by Final State. Provide the formal construction to convert it to a PDA M' that accepts by Empty Stack.

Solution:

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$. We construct $M' = (Q \cup \{p_0, p_e\}, \Sigma, \Gamma \cup \{X_0\}, \delta', p_0, X_0, \emptyset)$, where $p_0, p_e \notin Q$ and $X_0 \notin \Gamma$.

The transition function δ' is defined as:

1. $\delta'(p_0, \epsilon, X_0) = \{(q_0, Z_0 X_0)\}$ (Start M with a marker X_0 below Z_0)
2. For all $q \in Q, a \in \Sigma \cup \{\epsilon\}, Y \in \Gamma, \delta'(q, a, Y)$ includes $\delta(q, a, Y)$.
3. For all $q \in F$, and $Y \in \Gamma \cup \{X_0\}$, add ϵ -transitions to the eraser state: $(p_e, Y) \in \delta'(q, \epsilon, Y)$.
4. For all $Y \in \Gamma \cup \{X_0\}, \delta'(p_e, \epsilon, Y) = \{(p_e, \epsilon)\}$ (Pop everything).

Practice Problem 3.6. Are Deterministic Context-Free Languages (DCFLs) closed under complementation? Briefly explain.

Solution:

Yes, DCFLs are closed under complementation. Since a DPDA has exactly one valid move for any input configuration, it behaves somewhat like a DFA.

By carefully swapping the accepting and non-accepting states (while handling infinite ϵ -loops appropriately), a DPDA can be constructed to accept the complement of the language.

This stands in stark contrast to NPDAs (and general CFLs), which are *not* closed under complementation.